Define Relationships Attributes

Now we are finally in one of the most exciting parts of **SQLModel**.

Relationship Attributes. ✨

We currently have a `team` table:

id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

And a `hero` table:

id	name	secret_name	age	team_id
1	Deadpond	Dive Wilson	null	2
2	Rusty-Man	Tommy Sharp	48	1
3	Spider-Boy	Pedro Parqueador	null	1

Now that you know how these tables work underneath and how the model classes represent them, it's time to add a little convenience that will make many operations in code simpler.

Declare Relationship Attributes

Up to now, we have only used the `team_id` column to connect the tables when querying with `select()`:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

This is a **plain field** like all the others, all representing a **column in the table**.

But now let's add a couple of new special attributes to these model classes, let's add `Relationship` attributes.

First, import `Relationship` from `sqlmodel`:

Python 3.10+

```
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Next, use that `Relationship` to declare a new attribute in the model classes:

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

What Are These Relationship Attributes

These new attributes are not the same as fields, they **don't represent a column** directly in the database, and their value is not a singular value like an integer. Their value is the actual **entire object** that is related.

So, in the case of a `Hero` instance, if you call `hero.team`, you will get the entire `Team` instance object that this hero belongs to. ✨

For example, you could check if a `hero` belongs to any `team` (if `.team` is not `None`) and then print the team's `name`:

```
if hero.team:
    print(hero.team.name)
```

Relationship Attributes or `None`

Notice that in the `Hero` class, the type annotation for `team` is `Team | None`.

This means that this attribute could be `None`, or it could be a full `Team` object.

This is because the related **`team_id` could also be `None`** (or `NULL` in the database).

If it was required for a `Hero` instance to belong to a `Team`, then the `team_id` would be `int` instead of `int | None`, its `Field` would be `Field(foreign_key="team.id")` instead of `Field(default=None, foreign_key="team.id")` and the `team` attribute would be a `Team` instead of `Team | None`.

Relationship Attributes With Lists

And in the `Team` class, the `heroes` attribute is annotated as a list of `Hero` objects, because that's what it will have.

SQLModel (actually SQLAlchemy) is smart enough to know that the relationship is established by the `team_id`, as that's the foreign key that points from the `hero` table to the `team` table, so we don't have to specify that explicitly here.

> **Tip**
>
> There's a couple of things we'll check again in some of the next chapters, about the `list["Hero"]` and the `back_populates`.
>
> But for now, let's first see how to use these relationship attributes.

Next Steps

Now let's see some real examples of how to use these new **relationship attributes** in the next chapters. ✨